# Module 11 - File Handling Advanced

---

## Table of Contents


- [ ] 1. CSV Files — Reading and Writing Structured Data
- [ ] 2. JSON Files — Config and API Response Files
- [ ] 3. File Encoding — Why Your Log Parser Will Break Without This
- [ ] 4. `pathlib` — The Modern Path Toolkit
- [ ] 5. Creating, Moving, and Deleting Files Safely
- [ ] 6. `os.walk()` for Log Aggregation
- [ ] 7. Summary and Next Steps


---

## 1. CSV Files — Reading and Writing Structured Data

CSV (Comma-Separated Values) is the most common format for exporting vulnerability scanners, SIEM alert exports, asset inventories, and threat intelligence feeds.

You could parse CSV manually with `split(',')` — but this breaks the moment a field contains a comma (e.g. `"Apache HTTP Server, version 2.4"`).

Python's built-in `csv` module handles all edge cases correctly: quoted fields, commas inside fields, different delimiters, header rows.

```
import csv

# Reading
with open('file.csv', 'r', newline='') as f:
    reader = csv.DictReader(f)   # treats first row as header -> each row
is a dict
    for row in reader:
        print(row['column_name'])

# Writing
with open('output.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=['col1', 'col2'])
    writer.writeheader()
    writer.writerow({'col1': 'value1', 'col2': 'value2'})
```

### Why `newline=''`?

The `csv` module handles its own line endings. If you don't pass `newline=''`, Python's universal newline translation will interfere and you may get blank lines between rows on Windows. Always include it.

In [ ]:
```
# Topic: csv.DictReader — parse a vulnerability scan export

import csv

unpatched_critical = []

with open("data/vulnerabilities.csv", "r", newline="") as vuln_file:
    reader = csv.DictReader(vuln_file)   # first row becomes the key names

    for row in reader:
        # Each row is a dict: {'cve_id': '...', 'cvss_score': '...', ...}
        cvss = float(row["cvss_score"])  # DictReader returns strings — convert!
        is_patched = row["is_patched"].lower() == "true"

        if cvss >= 9.0 and not is_patched:
            unpatched_critical.append({
                "cve_id": row["cve_id"],
                "cvss_score": cvss,
                "system": row["affected_system"]
            })

print(f"Unpatched critical vulnerabilities: {len(unpatched_critical)}")
for vuln in unpatched_critical:
    print(f"  [{vuln['cvss_score']}] {vuln['cve_id']} — {vuln['system']}")
```

In [ ]:
```
# Topic: csv.DictWriter — write a filtered report back to CSV

import csv

output_path = "data/critical_unpatched.csv"

fieldnames = ["cve_id", "cvss_score", "system"]

with open(output_path, "w", newline="") as out_file:
    writer = csv.DictWriter(out_file, fieldnames=fieldnames)
    writer.writeheader()                  # writes the column name row

    for vuln in unpatched_critical:
        writer.writerow(vuln)             # each dict becomes one row

print(f"Written to {output_path}")

# Verify
with open(output_path, "r") as f:
    print(f.read())
```

## 🎯 Practice — CSV

**Exercise 1 — Write**

Read `data/vulnerabilities.csv`. Build two lists:

- `patched_vulns`: all vulnerabilities where `is_patched` is `true`
- `high_risk_unpatched`: all where `cvss_score >= 7.0` AND `is_patched` is `false`

For each list, store only the `cve_id` and `cvss_score` as a dict. Print the count of each list.

In [ ]:
```
# Your code here
import csv
```

▶ 💡 **SOLUTION**

---

## 2. JSON Files — Config and API Response Files

JSON is the standard format for:

- Security tool configuration files
- API responses (VirusTotal, Shodan, MISP, etc.)
- SIEM rule exports
- Threat intelligence (STIX/TAXII uses JSON)

Python's `json` module converts between JSON text and Python dicts/lists:

| Function | Direction | Use |
|---|---|---|
| `json.load(file)` | JSON file → Python dict | Reading config/data files |
| `json.loads(string)` | JSON string → Python dict | Parsing API responses |
| `json.dump(data, file)` | Python dict → JSON file | Writing configs/reports |
| `json.dumps(data)` | Python dict → JSON string | Serialising for APIs |

The `indent` parameter makes output human-readable: `json.dump(data, f, indent=2)` — always use this when humans will read the file.

In [ ]:
```
# Topic: json.load() — reading a security tool config file

import json

with open("data/config.json", "r") as config_file:
    config = json.load(config_file)   # returns a Python dict

# Access nested values with dict syntax
print("=== SIEM Configuration ===")
print(f"  Host:     {config['siem']['host']}")
print(f"  Port:     {config['siem']['port']}")
```

```
print(f"  Enabled:  {config['siem']['enabled']}")

print("\n=== Firewall Configuration ===")
print(f"  Default policy: {config['firewall']['default_policy']}")
print(f"  Allowed ports:  {config['firewall']['allowed_ports']}")

print("\n=== Alerting ===")
print(f"  Critical threshold: {config['alerting']['critical_threshold']}")
print(f"  Email on critical:  {config['alerting']['email_on_critical']}")
```

In [ ]:

```
# Topic: json.dump() — updating a config and writing it back

import json

# Load the existing config
with open("data/config.json", "r") as f:
    config = json.load(f)

# Make updates
config["firewall"]["allowed_ports"].append(8443)   # add HTTPS-alt
config["firewall"]["blocked_ips"] = [
    "185.220.101.47",  # TOR exit node from our blocklist
    "45.33.32.156",
]
config["retention"]["log_days"] = 180  # extend retention for compliance

# Write back — indent=2 makes it human-readable
with open("data/config.json", "w") as f:
    json.dump(config, f, indent=2)

print("Config updated.")

# Verify the write
with open("data/config.json", "r") as f:
    print(f.read())
```

## 🎯 Practice — JSON

**Exercise 2 — Write**

Read `data/config.json`. Check whether port 22 (SSH) is in `firewall.allowed_ports`.

Then build a new dict `scan_results` with the following structure and write it to `data/scan_results.json` with `indent=2`:

```
{
  "scan_id": "SCAN-2024-001",
  "target": "10.0.0.0/24",
  "open_ports": [22, 80, 443, 3389],
  "critical_findings": 2,
  "completed": True
}
```

Then read it back and print the `open_ports` list.

In [ ]:
```
# Your code here
import json
```

▶ 💡 **SOLUTION**

---

# 3. File Encoding — Why Your Log Parser Will Break Without This

When Python opens a file in text mode, it decodes bytes to strings using an encoding. The default encoding depends on the OS — on Windows it might be `cp1252`, on Linux `utf-8`.

This matters enormously in security work because:

- Logs generated by Windows systems often use `cp1252` or `latin-1`
- Linux tools generate `utf-8`
- Malware may use unusual encodings to evade detection
- A single bad character in a log file will crash your parser with `UnicodeDecodeError`

## Always specify encoding explicitly

```
# Risky: uses the OS default, which differs between machines
with open('logfile.txt', 'r') as f:
    ...

# Safe: always explicit
with open('logfile.txt', 'r', encoding='utf-8') as f:
    ...
```

## Handling unknown or mixed-encoding files

```
# errors='replace': replace bad characters with the replacement character
U+FFFD
# errors='ignore':  silently drop undecodable bytes
# errors='strict':  raise UnicodeDecodeError (the default)
with open('suspicious.log', 'r', encoding='utf-8', errors='replace') as f:
    content = f.read()
```

For forensic work, use `errors='replace'` rather than `errors='ignore'` — you want to know that bad bytes existed, even if you can't read them.

In [ ]:
```
# Topic: encoding — safe log parsing with explicit encoding

# Standard reading — specify utf-8 always
with open("data/access.log", "r", encoding="utf-8") as log_file:
    lines = log_file.readlines()

print(f"Read {len(lines)} lines from access.log (utf-8)")
```

```python
# Handling a file with potentially mixed encoding
# errors='replace' means bad bytes become the replacement character (U+FFFD)
# instead of crashing the whole parser
with open("data/access.log", "r", encoding="utf-8", errors="replace") as log_file:
    content = log_file.read()

# Check if any replacement characters slipped in (would indicate encoding issues)
replacement_char = "\ufffd"
if replacement_char in content:
    print("[WARN] Encoding issues detected — file may contain non-UTF-8 bytes")
else:
    print("[OK] No encoding issues detected")
```

---

## 4. `pathlib` — The Modern Path Toolkit

`pathlib` (introduced in Python 3.4) provides a cleaner, object-oriented way to work with paths. It replaces most of `os.path` with a more readable API.

```python
from pathlib import Path

p = Path('data/access.log')
p.exists()          # True/False
p.is_file()         # True/False
p.stem              # 'access'  (filename without extension)
p.suffix            # '.log'
p.name              # 'access.log'
p.parent            # Path('data')
p.stat().st_size    # file size in bytes
```

### Building paths with `/`

`pathlib` overloads the `/` operator for path joining — no more `os.path.join()`:

```python
data_dir = Path('data')
log_path = data_dir / 'logs' / 'access.log'   # clean and readable
```

### `os.path` vs `pathlib`

| Task | `os.path` | `pathlib` |
|---|---|---|
| Join paths | `os.path.join(a, b)` | `Path(a) / b` |
| Check exists | `os.path.exists(p)` | `p.exists()` |
| Get filename | `os.path.basename(p)` | `p.name` |
| Get extension | `os.path.splitext(p)[1]` | `p.suffix` |

| Task | `os.path` | `pathlib` |
|---|---|---|
| List directory | `os.listdir(p)` | `p.iterdir()` |
| Read entire file | manual `open()` | `p.read_text()` |

Both work. `os.path` is older and still common — `pathlib` is cleaner for new code.

In [ ]:
```
# Topic: pathlib — cleaner path operations for a security tool

from pathlib import Path

# Create Path objects
data_dir = Path("data")
log_file = data_dir / "access.log"     # / operator for path joining
config_file = data_dir / "config.json"

print("=== Path properties ===")
print(f"Full path:  {log_file}")
print(f"Name:       {log_file.name}")
print(f"Stem:       {log_file.stem}")    # 'access'
print(f"Suffix:     {log_file.suffix}")  # '.log'
print(f"Parent:     {log_file.parent}")
print(f"Absolute:   {log_file.resolve()}")

print("\n=== Existence checks ===")
print(f"exists:     {log_file.exists()}")
print(f"is_file:    {log_file.is_file()}")
print(f"is_dir:     {data_dir.is_dir()}")

print("\n=== File size ===")
print(f"size:       {log_file.stat().st_size} bytes")
```

In [ ]:
```
# Topic: pathlib iterdir() and glob() — find files by pattern

from pathlib import Path

data_dir = Path("data")

# List all files in the data directory
print("=== All files in data/ ===")
for item in sorted(data_dir.iterdir()):
    if item.is_file():
        print(f"  {item.name:25s} {item.stat().st_size:6d} bytes")

print("\n=== Only .log files (glob) ===")
for log_file in sorted(data_dir.glob("*.log")):
    print(f"  {log_file.name}")

print("\n=== Quick read with read_text() ===")
# pathlib shortcut — no need for open() for simple cases
first_line = (data_dir / "blocked_ips.txt").read_text(encoding="utf-8").split("\n")
print(f"  First line of blocked_ips.txt: {first_line}")
```

### 🎯 Practice — `pathlib`

**Exercise 3 — Write**

Using `pathlib`:

1. Create a `Path` object for the `data/` directory
2. Use `glob('*.txt')` to find all `.txt` files
3. For each file, print: the filename, size in bytes, and number of lines (use `read_text()` to get the content, then count `\n`)

Sort the output by file size (largest first).

In [ ]:
```
# Your code here
from pathlib import Path
```

▶ 💡 **SOLUTION**

---

## 5. Creating, Moving, and Deleting Files Safely

Beyond reading and writing, security tools often need to manage files: archiving processed logs, creating output directories, or cleaning up temp files.

| Operation | `os` module | `pathlib` |
|---|---|---|
| Create directory | `os.makedirs(path, exist_ok=True)` | `Path.mkdir(parents=True, exist_ok=True)` |
| Rename / move | `os.rename(src, dst)` | `Path.rename(dst)` |
| Delete file | `os.remove(path)` | `Path.unlink()` |
| Delete directory | `os.rmdir(path)` (must be empty) | `Path.rmdir()` (must be empty) |
| Copy file | `shutil.copy(src, dst)` | `shutil.copy(src, dst)` |

### ⚠️ Safety rule — always check before deleting

```
# BAD: crashes if file doesn't exist
os.remove('file.txt')

# GOOD: check first
path = Path('file.txt')
if path.exists():
    path.unlink()
```

In [ ]:
```
# Topic: creating directories and archiving processed logs

import os
import shutil
from pathlib import Path

# Create an archive directory for processed log files
archive_dir = Path("data") / "archive"
archive_dir.mkdir(parents=True, exist_ok=True)   # exist_ok prevents error if alrea
print(f"Archive directory: {archive_dir} (exists: {archive_dir.exists()})")

# Copy a processed log to the archive (keep the original)
source = Path("data") / "access.log"
destination = archive_dir / "access.log.bak"

shutil.copy(source, destination)
print(f"Archived: {source.name} -> {destination}")
print(f"Archive size: {destination.stat().st_size} bytes")

# List the archive
print("\nArchive contents:")
for item in archive_dir.iterdir():
    print(f"  {item.name}")
```

---

## 6. `os.walk()` for Log Aggregation

Real-world log infrastructure stores logs in nested directories — by date, by service, by host.

```
logs/
  nginx/
    2024-03-14/access.log
    2024-03-15/access.log
  auth/
    2024-03-14/auth.log
    2024-03-15/auth.log
  firewall/
    2024-03-15/fw.log
```

`os.walk()` lets you recursively walk this entire structure and process every log file. This is the foundation of any log aggregation script.

In [ ]:
```
# Topic: os.walk() — aggregate all log content from nested directories

import os

# Create a nested log structure to demonstrate
os.makedirs("data/logs/webserver", exist_ok=True)
os.makedirs("data/logs/auth", exist_ok=True)
```

```
with open("data/logs/webserver/access.log", "w") as f:
    f.write("2024-03-15 09:00:01 | 10.0.0.5 | GET /index.html | 200\n")
    f.write("2024-03-15 09:01:12 | 185.220.101.47 | POST /login | 401\n")

with open("data/logs/auth/auth.log", "w") as f:
    f.write("2024-03-15 09:01:12 Failed password for user admin from 185.220.101.47
    f.write("2024-03-15 09:05:00 Accepted password for alice from 10.0.0.14\n")

# Now walk and aggregate all .log files
all_log_lines = []

for dirpath, subdirs, files in os.walk("data/logs"):
    for filename in files:
        if filename.endswith(".log"):
            full_path = os.path.join(dirpath, filename)
            with open(full_path, "r", encoding="utf-8") as log_file:
                for line in log_file:
                    all_log_lines.append(line.strip())

print(f"Total log lines aggregated: {len(all_log_lines)}")
for line in all_log_lines:
    print(f"  {line}")
```

## 🎯 Practice — Log Aggregation

**Exercise 4 — Write**

Write a function `aggregate_logs(root_dir, extension)` that:

- Uses `os.walk()` to recursively find all files with the given extension
- Reads each file line by line (skip empty lines)
- Returns a list of tuples: `(source_filepath, line_content)`

Call it with `aggregate_logs('data/logs', '.log')` and print:

- Total lines found
- The source file for each line

In [ ]:
```
# Your code here
import os
```

▶ 💡 **SOLUTION**

---

# 7. Summary and Next Steps

## Advanced File Handling — Quick Reference

| Tool | Use case |
|---|---|
| `csv.DictReader` | Parse vulnerability exports, asset lists, alert CSVs |
| `csv.DictWriter` | Write structured reports or filtered data |
| `json.load()` | Load config files, API responses |
| `json.dump()` | Write updated configs, scan results |
| `encoding='utf-8'` | Always specify — prevents cross-platform breaks |
| `errors='replace'` | Safe parsing of logs with unknown encoding |
| `pathlib.Path` | Modern, readable path manipulation |
| `Path.glob('*.log')` | Find files by pattern |
| `os.makedirs(exist_ok=True)` | Create output dirs without crashing |
| `os.walk()` | Recursively process log directories |

## Next Steps

**02_Drills_File_Handling.ipynb** — 15 exercises across all file handling topics